Development Methodology and Architecture Assessment for a Novelty Similarity Detection Model

Alisher Tokesh | CSE-2507M

Assignment 2

This report continues Assignment 1 and assesses whether the recommended development methodology and architecture fit the same novelty similarity detection pilot. The analysis keeps the 17-week critical path, USD 43,916 budget, human review policy, and quality targets established in the preceding project plan.

Main conclusion. A stage-gated iterative R&D approach and a modular monolith with explicit adapters are appropriate for the pilot. They preserve the ability to calibrate the multilingual model and validate reviewers feedback, while preventing budget and integration complexity from outrunning the evidence needed for a production decision.

Table of Contents

1 Project Analysis 3
1.1 Goals Requirements and Context 3
1.2 Constraints Risks and Priorities 4
2 Development Methodology Assessment 5
2.1 Selected Method and Fit 5
2.2 Method Risks and Alternatives 6
3 Architectural Approach Assessment 7
3.1 Architectural Style and Alignment 7
3.2 Quality Attributes and Adjustments 8
4 Conclusions and Recommendations 9
References 9

Continuation from Assignment 1

Assignment 1 defined the novelty model as a controlled decision-support service rather than an autonomous approval system. Its scope includes a cleaned historical corpus, threshold calibration, FastAPI with pgvector search, a coordinator workflow, quality monitoring, pilot training, and closure. The present report evaluates the development decisions required to deliver those outputs within the approved baseline.

Project baseline. The planned implementation lasts 17 weeks. The direct cost is USD 38,188 with a 15 percent contingency reserve of USD 5,728, giving a total budget of USD 43,916. These values are treated as constraints for methodology and architecture decisions, not merely as financial background.

1 Project Analysis

1.1 Goals Requirements and Context

The project aims to identify duplicate and near-duplicate thesis topics before final academic approval. It accepts a title, abstract, keywords, domain and language, compares the proposal with historical topics, and returns a novelty score, similarity evidence, and a decision label. Reviewers remain responsible for the final approval, especially when a topic is classified as high risk.

Category	Priority	Requirement	Evidence of completion
Functional	Must	Load a cleaned historical corpus and calculate top-k similar topics.	Versioned corpus and reproducible search result.
Functional	Must	Return score components, label and similar topics through FastAPI.	API contract and reviewer-readable explanation.
Functional	Should	Support reviewer confirmation or override with a reason.	Auditable decision record.
Functional	Should	Provide a coordinator workflow and quality dashboard.	Pilot users can inspect cases and metrics.
Nonfunctional	Must	Protect topic data and avoid automatic rejection.	Role-based access and human-in-the-loop policy.
Nonfunctional	Must	Produce reproducible calibration and evaluation results.	Dataset, model and threshold versions are recorded.
Nonfunctional	Should	Meet response and quality targets under pilot load.	KPI report and load-test evidence.

Table 1. Prioritized requirements derived from the Assignment 1 scope.

1.2 Constraints Risks and Priorities

The project is an applied ML and integration pilot, so its principal uncertainty is not only engineering effort. Historical data may be incomplete, language coverage may be uneven, and a technically high similarity score may still require contextual academic judgement. The schedule gives only two weeks of slack to corpus preparation and hybrid calibration; the API and workflow integration stream is critical.

Constraint or risk	Effect on project	Required control
17-week critical path	A delay in approvals, backend integration or pilot readiness delays closure.	Stage gates and weekly critical-path review.
Budget USD 43,916	Premature service separation or excess experimentation consumes the reserve.	Release reserve only by approved change request.
Inconsistent historical data	Noisy similarity and weak keyword statistics reduce trust.	Data audit, minimum fields and sampled label review.
Kazakh Russian English coverage	Uneven language quality can create unfair results.	Language-tagged validation sets and multilingual embeddings.
False high-risk flags	Students can lose trust if a flag is treated as a decision.	Mandatory reviewer review, appeal path and threshold tuning.
Privacy and access controls	Approval delays can block model work and pilot use.	Early data-owner approval, restricted logging and RBAC.

Table 2. Project constraints and risk controls inherited from Assignment 1.

Pilot Quality Targets

The architecture and process are accepted only if they support duplicate-detection F1 of at least 0.82, near-duplicate recall of at least 0.75, top-k latency of no more than 1.5 seconds, at least 25 percent review-time saving, user acceptance of at least 4.0 out of 5, and a false high-risk rate no greater than 8 percent.

2 Development Methodology Assessment

2.1 Selected Method and Fit

The proposed method is stage-gated iterative R&D delivery: short Scrum-style planning and review cycles for software and model work, a Kanban board with work-in-progress limits for data remediation, and formal gates at points where evidence or funding must be released. It is not a full bureaucratic waterfall. The gates protect the schedule, budget and academic-risk controls that Assignment 1 already established.

Period	Increment and decision gate	Primary evidence
Weeks 1 to 3	Scope, data access and baseline preparation	Data approval, duplicate policy, labelled-pair protocol.
Weeks 3 to 6	Baseline model and corpus work	Reproducible hashing baseline and corpus quality report.
Weeks 5 to 11	Multilingual embedding, pgvector and API integration	Top-k retrieval, service contract and versioned configuration.
Weeks 11 to 13	Quality security and performance validation	F1, recall, latency, security and error analysis.
Weeks 13 to 15	Coordinator pilot	Reviewer decisions, disagreement analysis and acceptance feedback.
Weeks 15 to 17	Deployment training and closure	Rollout decision, training record and lessons learned.

Table 3. Iterative delivery aligned with the 17-week CPM baseline.

Why the fit is strong. Model thresholds, corpus quality and reviewer expectations cannot be fixed with confidence at the start. Iterations expose those uncertainties early; gates stop the team from treating a working API as proof that the decision-support service is trustworthy.

2.2 Method Risks and Alternatives

Criterion	Stage-gated iterative R&D	Waterfall	Kanban only
Data and model uncertainty	High fit: baseline and calibration can change from evidence.	Low fit: feedback arrives after detailed commitments.	Medium fit: flexible but needs quality gates.
17-week deadline	High fit: time boxes expose delays on critical activities.	Medium fit: predictable but inflexible around evidence.	Medium fit: flow may hide fixed milestones.
Budget reserve control	High fit: funding and reserve link to gate evidence.	High fit: approval controls are simple.	Low fit unless added financial controls exist.
Reviewer involvement	High fit: regular demos and label reviews.	Low fit: reviewers see the product late.	Medium fit: possible but not time-boxed.
Operations after pilot	Medium fit: switch to Kanban after release.	Low fit: change requests become slow.	High fit: suitable for maintenance.

Table 4. Comparative methodology assessment.

Methodology Risks and Responses

The chosen method can fail if experimentation has no stopping rule, if reviewers miss every review, or if the data-remediation queue grows without a limit. The response is to freeze a small benchmark before calibration, assign a named academic decision delegate, apply Kanban WIP limits, and treat a change that exceeds one iteration as a scoped research decision. A limited waterfall phase is justified only for privacy approval and procurement documents whose acceptance criteria are stable.

Recommendation. Use Kanban as the operational control within the iterative method, not as a replacement for sprint reviews, pilot gates and budget authority.

3 Architectural Approach Assessment

3.1 Architectural Style and Alignment

The existing repository is already organized like a modular monolith: preprocessing, embeddings, keyword analysis, configuration, schemas, scoring, CSV I/O, CLI and FastAPI are separated. The EmbeddingBackend abstraction supports both a deterministic hashing fallback and a SentenceTransformer backend. The present prototype ranks in process memory, while the Assignment 1 plan reserves pgvector for production-style similarity search.

The recommended target remains a modular monolith with hexagonal boundaries. The assessment application service owns the use case; the hybrid scorer and policy remain independent of infrastructure; adapters connect the service to FastAPI, the topic repository, the embedding provider, audit logging, metrics and background evaluation. This is more proportionate than microservices for the 17-week, USD 43,916 pilot.

Coordinator or reviewer	FastAPI adapter	Assessment service	Hybrid scoring policy
Reviewer decision and audit	Topic repository port	Embedding provider port	Configuration registry
PostgreSQL with pgvector	Multilingual embedding model	Metrics and quality dashboard	Background evaluation jobs

Figure 1. Target modular monolith with adapters and operational boundaries.

3.2 Quality Attributes and Adjustments

Attribute	Current support	Required adjustment
Reliability	Unit tests cover duplicate and unrelated cases; health endpoint exists.	Add integration tests for storage, restart recovery and malformed corpus data.
Performance	In-memory search is simple; pgvector HNSW schema is planned.	Use indexed pgvector retrieval, batch embedding and measured p95 latency.
Security and privacy	No application authentication or audit adapter is visible.	Add platform RBAC, TLS, least-privilege DB role and redacted logs.
Explainability	Score components and similar topics are returned by the model.	Persist configuration, candidates and reviewer rationale with each assessment.
Maintainability	Modules and embedding abstraction isolate many changes.	Formalize repository, audit and provider ports; record ADRs and API versions.
Reproducibility	Configurable weights, thresholds and deterministic fallback exist.	Version corpus, embedding model, threshold report and benchmark together.

Table 5. Architecture alignment with nonfunctional requirements and KPI targets.

Justified Architectural Adjustments

First, replace service-owned topic lists with a TopicRepository port and a pgvector implementation, while retaining scoring in the domain layer. Second, make the audit record a first-class adapter that stores corpus version, model version, thresholds, score components, returned candidates and reviewer action. Third, move ingestion, batch embedding and evaluation into controlled background jobs so long-running work cannot delay the assessment endpoint. Microservice extraction should wait until independent scaling or a separate security boundary produces a measured benefit.

4 Conclusions and Recommendations

The selected methodology and architecture fit the same project baseline defined in Assignment 1. The 17-week critical path and USD 43,916 limit require a small-team design with visible evidence gates. At the same time, the multilingual novelty model cannot be safely completed through a rigid sequential plan because data quality, threshold calibration and reviewer interpretation are uncertain. Stage-gated iterative R&D therefore balances flexibility with funding and academic governance.

A modular monolith with hexagonal adapters is the appropriate architectural choice. It uses the existing repository boundaries, enables a pgvector transition without coupling the scoring policy to a database, and preserves a single deployable service for a constrained pilot. The important future investments are persistent corpus management, human-decision auditability, access control, benchmark governance and operational monitoring, rather than premature microservices.

Prioritized Recommendations

Timing	Recommendation	Decision criterion
Weeks 1 to 3	Approve policy, access, benchmark and error definitions.	No model calibration before data and review rules are accepted.
Weeks 3 to 11	Deliver a vertical slice with multilingual embeddings, pgvector and API.	Top-k evidence is reproducible and below the latency target.
Weeks 11 to 15	Add audit trail, RBAC, test evidence and reviewer pilot.	F1, recall, false-high-risk rate and user acceptance meet targets.
Weeks 15 to 17	Deploy only after the pilot gate; otherwise use reserve for correction.	Production decision is evidence-based, not deadline-based.

Table 6. Recommendations linked to the original schedule and quality gates.

References

1. Assignment 1 Alisher Tokesh. Project Management and Financial Justification for a Novelty Similarity Detection Model. Course report.

2. Hybrid Thesis Topic Novelty Assessment Model. README.md and source modules in novelty-similarity-detection-model.

3. Assignment 2 brief supplied by the course. Methodology and architectural approach assessment.